Real-Time, Stability-Aware Customer Segmentation and Personalized WhatsApp Campaign Recommendation

Sarthak Gawari, Yash Kachale, Sushant Bhanwase, and Jaykumar Kale
Department of Information Technology
Pune Institute of Computer Technology (PICT), Pune, India
B.E. Information Technology Final-Year Project

***Abstract*—This paper presents a research plan for extending an existing customer segmentation and marketing recommendation platform with real-time profile updates and stability-aware decisions. The implemented baseline validates CSV transactions, computes customer-level Recency, Frequency, and Monetary (RFM) features, selects a K-Means model through internal clustering metrics, generates data-driven segment labels, and applies consent, suppression, recency, and rolling frequency-cap rules before campaign recommendation. On the repository's bundled 57-row synthetic verification dataset, the current reproducible output retains 52 rows for analysis, represents 15 customers, and selects K=3 with a silhouette score of 0.4562. These results describe the batch baseline only. Event-driven ingestion, incremental clustering, segment-stability measurement, recommendation-stability checks, persistent event processing, and latency evaluation are proposed extensions and have not yet been measured. The planned study compares batch processing with event-driven updates without and with a stability-assessment layer, while treating eligibility safeguards as mandatory constraints.**

***Index Terms*—RFM analysis, customer segmentation, K-Means, streaming clustering, decision stability, campaign recommendation, marketing fatigue, consent-aware systems.**

I. Introduction

Retail transaction systems commonly support periodic customer analysis, but a batch workflow can delay the effect of a new purchase on a customer's profile and campaign decision. A more responsive system must update relevant customer features when a transaction arrives, while avoiding unstable segment changes caused by small disturbances or repeated model recalculation. Responsiveness is also insufficient if the resulting recommendation violates consent, suppression, or contact-frequency constraints.

This work starts from an implemented batch platform for customer segmentation and personalized WhatsApp campaign recommendation. The baseline performs transaction validation, RFM feature engineering, K-Means clustering, dynamic segment labelling, explainable campaign mapping, and policy checks. The proposed research extension investigates whether an event-driven update path combined with an explicit stability-assessment layer can improve decision reliability without imposing excessive processing latency.

The contributions planned by this study are:

- a reproducible batch baseline for RFM segmentation and eligibility-aware recommendation;
- a practical comparison of batch, event-driven, and stability-aware update strategies;
- operational definitions for segment drift, recommendation switching, update latency, and policy violations; and
- an implementation roadmap that preserves mandatory consent and fatigue safeguards while adding incremental research functionality.

No claim is made that the proposed extension is already implemented or that the hypotheses have been proven.

II. Problem Statement and Research Objectives

Let T denote a stream of transaction events and let c be a customer. A new event may change c's RFM profile, cluster assignment, and campaign recommendation. The research question is:

> Can a stability-aware decision layer improve the reliability of real-time RFM-based customer segmentation and personalized campaign recommendations while maintaining low processing latency and enforcing marketing eligibility constraints?

The planned hypotheses are:

- H1: event-driven profile and recommendation updates have lower event-to-decision latency than the existing batch workflow;
- H2: stability checks reduce unnecessary segment and recommendation switching under controlled transaction-data disturbances; and
- H3: consent, suppression, and frequency-cap guarantees remain satisfied after the real-time extension.

The objectives are to establish a reproducible baseline, implement the smallest feasible event or micro-batch prototype, measure stability and latency, and report limitations without treating replayed historical data as a live retail integration.

III. Related Work

Hughes introduced RFM as a practical behavioural framework for direct marketing [1]. MacQueen's K-Means formulation provides the centroid-based clustering foundation used by the baseline [2]. Rousseeuw's silhouette coefficient [3], the

CSV upload → validation and cleaning → RFM features → K-Means and metric sweep → dynamic segment labels → explainable recommendation → consent, suppression, and frequency-cap gate → simulated WhatsApp preview

Fig. 1. Implemented batch baseline workflow.

Davies–Bouldin index [4], and the Calinski–Harabasz criterion [5] provide complementary internal measures of cluster structure. Chen, Sain, and Guo applied RFM-based data mining to online retail customer segmentation [6], while Fader, Hardie, and Lee studied transaction-frequency models for customer valuation [7].

Streaming and incremental clustering motivate the proposed update strategy. Bifet and Kirkby describe adaptive stream-mining methods for evolving data [11], and Aggarwal et al. present a framework for clustering data streams under limited memory [12]. These works address online data analysis, but do not by themselves define consent-aware campaign dispatch decisions. Cluster validity and temporal stability are related to the use of Adjusted Rand Index (ARI) for comparing partitions [13].

Marketing interaction frequency also requires governance. Godfrey, Seiders, and Voss studied customer fatigue arising from repeated service interactions [8]. Kumar and Reinartz discuss customer relationship strategy and contact management [9], while Verhoef, Kannan, and Inman describe the complexity of coordinated omnichannel customer interactions [10]. The closest distinction of this study is therefore not a claim of first discovery: it is an applied empirical evaluation of the joint behaviour of event-driven RFM updates, stability checks, explainable recommendations, and hard eligibility gates in one auditable prototype.

IV. Existing Baseline Architecture

The current implementation is a FastAPI and React application. The active workflow is process-local: a CSV upload is cleaned into an in-memory session, an analysis request computes RFM and clustering, and subsequent REST requests expose the results to the dashboard. SQLAlchemy models, tenant query helpers, WhatsApp signature parsing, webhook deduplication logic, a LinUCB implementation, and Shopify/WooCommerce parsers exist in the repository, but they are not fully wired into the active upload-to-recommendation path.

The baseline excludes cancellation invoices from RFM, counts distinct invoice identifiers for frequency, and computes monetary value from quantity multiplied by unit price. The reference date is one day after the latest valid invoice date. Monetary values are transformed with $\log(1 + x)$ and the implementation uses `RobustScaler`; the paper does not describe this as Z-score standardization.

V. Implemented Baseline Methodology

For customer c_i, the baseline computes:

$$R_i = t_{ref} - \max(t_{ij}), \tag{1}$$

$$F_i = |\{\text{unique invoice identifiers for } c_i\}|, \tag{2}$$

$$M_i = \sum_j q_{ij} p_{ij}. \tag{3}$$

Here, t_{ref} is the latest valid transaction date plus one day. Rows with missing identifiers, invalid dates, non-positive quantities, or non-positive unit prices are rejected. Cancellation invoices are recorded separately and excluded from the RFM input.

K-Means minimises the within-cluster objective:

$$J = \sum_{k=1}^{K} \sum_{z_i \in S_k} \|z_i - \mu_k\|^2. \tag{4}$$

The implementation evaluates feasible values of K and selects the maximum silhouette score. Davies–Bouldin and Calinski–Harabasz values are reported for comparison. Segment labels are generated from cluster statistics rather than fixed cluster identifiers.

The recommendation engine maps segment characteristics to campaign types and records a human-readable explanation. Before an eligible recommendation can be previewed, the fatigue service checks explicit consent, suppression status, a default maximum of three messages in a rolling 30-day window, and campaign-specific recency conditions. These checks are treated as hard constraints in the proposed real-time extension.

A. Transaction Data Contract and Quality Gates

The ingestion service accepts a comma-separated transaction file. Five fields are required: invoice identifier, customer identifier, invoice date, quantity, and unit price. Header aliases are supported for common retail exports, including `InvoiceNo`, `CustomerID`, `InvoiceDate`, `Quantity`, and `UnitPrice`. Optional columns carry consent, suppression, last-contact, and recent-message metadata. Normalisation is applied to headers before validation, so a compatible export does not need to use the internal snake-case names.

The quality gate returns a cleaning summary rather than silently removing data. The summary includes original and retained row counts, cancellation count, each rejection category, duplicate count, and sample validation issues. A cancellation is retained for audit information but excluded from RFM aggregation. Exact duplicate lines are removed using the transaction identity fields defined by the ingestion service. This design allows the dashboard to show both the usable analytical population and the reasons that records did not enter the model.

TABLE I
BASELINE TRANSACTION DATA CONTRACT

Field	Type	Validation or use
Invoice ID	String	Required; groups line items and identifies cancellations when prefixed by C.
Customer ID	String	Required; rows without an identifiable customer are rejected.
Invoice date	Date/time	Required; unparseable dates are rejected.
Quantity	Numeric	Required; values less than or equal to zero are rejected.
Unit price	Numeric	Required; values less than or equal to zero are rejected.
Consent / suppression	Boolean	Optional metadata used by eligibility checks.
Messages in 30 days	Integer	Optional metadata used by the rolling frequency cap.

TABLE II
IMPLEMENTED INTERNAL CLUSTERING METRICS

Metric	Interpretation
Silhouette	Compares within-cluster cohesion with the nearest other cluster; higher values are preferred.
Davies–Bouldin	Average similarity to the most similar competing cluster; lower values are preferred.
Calinski–Harabasz	Between-cluster dispersion relative to within-cluster dispersion; higher values are preferred.

B. RFM Scoring and Feature Preparation

The clustering feature vector is distinct from the ordinal score shown to users. Recency is measured in days, where a smaller value indicates a more recent purchase. Frequency is the number of distinct invoices, not the number of CSV rows. Monetary value is the sum of line totals for valid, non-cancellation transactions. The reference date is calculated per run as one day after the latest valid invoice date, avoiding a zero recency for the most recent customer.

For ranking and explanation, each raw RFM value is converted into a quartile score from one to four. Recency uses a reversed ordering, while larger frequency and monetary values receive larger scores. The composite score is a documented heuristic:

$$Q_i = 0.3R_i^{score} + 0.3F_i^{score} + 0.4M_i^{score}. \tag{5}$$

The monetary component receives the largest weight because it is used as a ranking aid for campaign prioritisation; this score is not a trained response or revenue prediction model. For small samples or tied values where four unique quantile bins cannot be formed, the implementation uses a rank-based fallback.

Before K-Means, monetary value is transformed with $\log(1 + M)$ and the three-dimensional matrix $[R, F, \log(1 + M)]$ is fitted with `RobustScaler`. Robust scaling uses median and interquartile-range statistics, reducing the influence of extreme retail spend values. This is an implementation detail that differs from standard Z-score scaling and is important for reproducibility.

C. Clustering Procedure and Edge Cases

The clustering service uses scikit-learn K-Means with a configurable range, a fixed default random seed of 42, and automatic selection by the highest available silhouette score. The feasible maximum is capped by the number of customer profiles. For a dataset with fewer than two customers, the service returns a clear validation error rather than producing a degenerate model. If a candidate assignment contains fewer than two distinct labels, silhouette, Davies–Bouldin, and Calinski–Harabasz values are recorded as unavailable for that candidate.

After the selected model is fitted, the service builds a profile for every cluster containing size, median recency, median frequency, median monetary value, and example customer identifiers. The label generator compares each cluster median with the dataset median. Consequently, a cluster number has no semantic meaning across independent runs; a label such as "Loyal High-Value" is derived from the observed statistics. This is useful for explainability but creates a requirement for label alignment in the proposed longitudinal experiments.

D. Recommendation Policy and Auditability

The recommendation engine first maps the dynamic segment label to a campaign archetype. Loyal or frequent high-value groups map to a loyalty reward, recent high-spenders to an upsell, inactive or at-risk groups to a win-back campaign, and low-engagement groups to re-engagement. The mapping is deterministic and can be inspected in the source code. The output does not claim that any campaign will produce a conversion.

For every customer, the engine returns the campaign type, display name, heuristic score, segment label, supporting RFM values, consent and suppression metadata, recent message count, eligibility flag, blocking reason, and a list of individual checks. The explanation explicitly states when engagement history is unavailable. This prevents missing response data from being silently converted into a probability or an effectiveness claim.

The ordering of these checks is intentional. Consent and suppression are policy constraints, not model features that can be traded against a higher predicted score. The proposed stability layer must execute before any final recommendation is accepted, but it cannot override a failed eligibility check.

TABLE III
CAMPAIGN RECOMMENDATION AND ELIGIBILITY BEHAVIOUR

Stage	Implemented behaviour
Campaign mapping	Segment statistics determine a campaign archetype and explanation.
Consent	A false consent value immediately blocks outreach.
Suppression	A suppressed customer is blocked before frequency evaluation.
Frequency cap	Three or more messages in 30 days blocks the customer by default.
Recency window	Loyalty, win-back, and re-engagement rules can apply campaign-specific recency checks.
Output	All passed and failed checks are returned for audit and dashboard display.

E. REST API and Dashboard Workflow

The baseline exposes a narrow REST workflow. `POST /api/v1/ingest/upload` accepts a CSV and returns the cleaning summary. `POST /api/v1/analysis/run` computes RFM and clustering for the current session. Separate read endpoints expose the RFM table, clustering metrics, segment profiles, and filterable recommendations. `POST /api/v1/campaigns/preview` supports a simulated campaign preview. The React dashboard polls `/health`, presents the ingestion quality gate, and then displays segments, metric cards, recommendation audit checks, and a simulated WhatsApp preview.

The current API is intentionally a baseline rather than a real-time event API. There is no request-level tenant identity, authentication middleware, public order-event route, persistent recommendation store, or public WhatsApp webhook route. The frontend preview is a simulation and does not dispatch a real message. These boundaries define the control condition for the research extension and prevent the paper from treating an interface demonstration as a deployed integration.

VI. PROPOSED EVENT AND STABILITY DESIGN

The smallest feasible extension should introduce an internal transaction-event contract before adding external infrastructure. An event represents a validated business fact and is separate from a recommendation decision. This separation permits duplicate events to be identified before they modify a customer profile and permits a recommendation to be deferred without discarding the transaction.

For each accepted event, the service should first verify the event schema and tenant ownership, then apply the same positive-value and cancellation rules as the CSV path. A duplicate event should be acknowledged without incrementing frequency or monetary value. An out-of-order event should be stored with its source timestamp and processed according to the selected replay policy. These cases are essential because a low-latency path that double-counts a purchase is not a reliable improvement over the batch baseline.

TABLE IV
PROPOSED TRANSACTION-EVENT RECORD

Field	Purpose
Event ID	Idempotency key supplied by the source or derived from a canonical transaction representation.
Tenant ID	Ownership context required before reading or updating customer data.
Customer ID	Customer whose transaction history and RFM state may change.
Invoice ID	Business transaction identity used in frequency aggregation.
Occurred at	Source event time, retained separately from processing time.
Received at	Server receipt time used for latency measurement.
Payload version	Supports controlled schema evolution during replay.
Processing status	Accepted, duplicate, rejected, processed, or deferred for review.

A. Stability Measurements

The stability layer will maintain the previous customer assignment and recommendation, together with the model run identifier. A segment-switch indicator for customer i between runs t and $t+1$ is:

$$\delta_i^{seg}(t) = \mathbb{1}[a_i(t) \neq a_i(t+1)]. \tag{6}$$

For a population of n customers, the segment switching rate is:

$$S_{seg} = \frac{1}{n}\sum_{i=1}^{n} \delta_i^{seg}(t). \tag{7}$$

The same construction is used for campaign recommendations, producing S_{rec}. Partition similarity will additionally be measured with ARI, which compensates for agreement expected by chance and does not require cluster identifiers to have the same numeric names. Because dynamic labels can change when medians move, the implementation must align clusters using centroid similarity before calculating longitudinal changes.

Centroid drift will be reported as the distance between aligned centroids after the same preprocessing configuration has been applied:

$$D_{centroid} = \frac{1}{K}\sum_{k=1}^{K} \|\mu_k(t+1) - \mu_k(t)\|_2. \tag{8}$$

The research prototype will evaluate thresholds for S_{seg}, S_{rec}, and $D_{centroid}$ rather than assuming that one fixed threshold is universally correct. A threshold must be selected on a training or calibration replay and evaluated on a separate temporal holdout to reduce threshold overfitting.

B. Decision States and Acceptance Criteria

Each event-driven decision should end in one of four auditable states: *accepted and eligible*, *accepted but ineligible*, *deferred because unstable*, or *rejected as invalid or duplicate*.

A deferred decision remains visible for review and cannot trigger a message. An ineligible decision may retain a campaign recommendation for analysis, but it cannot reach a dispatch adapter. This distinction allows the experiment to measure recommendation changes without weakening the safety gate.

The prototype will be considered ready for evaluation when it satisfies all of the following conditions: (1) the same event replay produces the same output under a fixed seed and configuration; (2) duplicate events do not alter customer frequency or monetary totals; (3) tenant ownership is checked before state mutation; (4) every decision records source time, processing time, previous state, candidate state, stability measurements, and eligibility checks; (5) unstable decisions are deferred rather than silently dispatched; and (6) the batch baseline remains runnable for comparison. These are engineering acceptance criteria, not achieved results.

C. Reproducibility and Verification Evidence

The repository fixes the default K-Means seed at 42 and stores generated empirical output in `docs/PROJECT_OUTPUTS.md`. The bundled dataset contains 57 raw rows and 15 customer profiles. The recorded cleaning output has 52 retained rows, two cancellations, five dropped rows, and one duplicate. The RFM reference date is 2024-10-06. The output selects $K = 3$ with a silhouette score of 0.4562.

The Python test suite covers ingestion, hand-checked RFM calculations, clustering boundaries and metrics, fatigue rules, recommendations, adapter parsing and deduplication, integration parsers, bandit behaviour, and isolated tenant queries. A local verification run on 22 October 2026 completed 106 tests. These unit and service tests establish component behaviour; they do not establish a complete persistent API-to-database-to-WhatsApp workflow. The paper therefore treats the test count as verification evidence for the implemented components, not as proof of production readiness.

VII. Research Methodology

The study will compare three conditions:

1) **A: Batch baseline.** Recompute the existing workflow after a batch of events or after a scheduled interval.
2) **B: Event-driven without stability checks.** Update the affected customer profile and recommendation using an incremental or micro-batch strategy, without suppressing changes based on stability.
3) **C: Event-driven with stability checks.** Apply the same update path, then compare the new assignment and recommendation with the prior state before accepting a material decision change.

Historical transactions may be replayed in timestamp order to simulate an event stream. Replay is a controlled simulation and must not be reported as a live retail integration. Each run will record a dataset hash, event order, preprocessing configuration, random seed, software versions, hardware, batch size, and model parameters.

TABLE V
Planned comparison and acceptance observations

Condition	Update strategy	Required observations
A	Existing batch workflow	Baseline latency, metrics, assignments, and recommendations
B	Event-driven without stability	Latency, throughput, switching, and policy outcomes
C	Event-driven with stability	Same measures plus stability overhead and deferred decisions

A practical prototype should begin with micro-batches or affected-customer recomputation. Incremental clustering can then be evaluated using `MiniBatchKMeans` or an equivalent strategy. The complete model should not be retrained for every individual event unless an experiment explicitly measures that condition as a comparison.

For a customer state s_t and a new event e_{t+1}, the proposed pipeline is:

$$s_{t+1} = \mathrm{UpdateRFM}(s_t, e_{t+1}), \tag{9}$$

followed by a candidate assignment a_{t+1} and recommendation r_{t+1}. A stability layer may accept the change only when the assignment difference, centroid drift, or recommendation change exceeds a defined materiality rule and the decision remains eligible. Exact thresholds are experimental parameters, not established results.

VIII. Evaluation Metrics and Experimental Design

The primary measurements are:

- event-to-profile-update latency and event-to-recommendation latency;
- throughput at increasing event loads and micro-batch sizes;
- segment switching rate and campaign recommendation switching rate;
- ARI or another partition similarity measure between consecutive assignments;
- centroid drift and recommendation changes under controlled perturbations;
- processing overhead introduced by stability assessment; and
- counts of consent, suppression, and frequency-cap violations.

Perturbation scenarios should include a normal purchase, repeated purchases for one customer, an isolated high-value transaction, an out-of-order event, duplicate delivery of the same event, and a transaction near a segment boundary. Each scenario should be repeated across fixed seeds and replay orders. The evaluation should include ablations for no stability layer, no fatigue gate, and no incremental update path.

TABLE VI
RECORDED K-MEANS BASELINE SWEEP ON THE BUNDLED DATASET

K	Silhouette	Davies–Bouldin	Calinski–Harabasz
2	0.4376	0.9209	10.25
3	**0.4562**	0.7150	13.42
4	0.4291	0.6505	15.59
5	0.3879	0.5166	13.53
6	0.3870	0.4231	13.90

IX. VERIFIED BASELINE RESULTS

The checked-in generated output identifies `backend/data/example_transactions.csv` as a 57-row bundled verification dataset. It contains 15 customers. The recorded cleaning output retains 52 rows, isolates two cancellation rows, drops five invalid rows, and reports one duplicate. The RFM reference date is 2024-10-06, with population medians of 144.0 recency days, 3.0 invoices, and 253.46 monetary units.

The recorded output selects $K = 3$ by silhouette score. These values are repository-recorded baseline outputs from the generated project-output artifact; they are not evidence for the proposed real-time hypotheses. The frequently repeated $K = 4$ values in older documents are excluded because their dataset and run provenance are not present in the checked-in repository.

X. IMPLEMENTATION TRACEABILITY AND SAFETY CONTROLS

The baseline was designed as a chain of small services so that each research measurement can be attached to a defined processing boundary. Ingestion produces a cleaned transaction frame and a summary; RFM produces one feature record per customer; clustering produces assignments and metric values; recommendations produce campaign decisions and policy checks. This separation makes it possible to compare an event-driven implementation against the same business rules rather than against an independently rewritten system.

The traceability boundary also identifies what cannot currently be inferred. A passing unit test for a tenant query does not prove that every router applies tenant filtering. A passing webhook parser does not prove that a delivery event updates a persistent campaign record. A simulated preview does not prove that the mock adapter was invoked. These distinctions are important for a research paper because component correctness and end-to-end system behavior are different claims.

A. Failure Modes in the Baseline

The baseline handles malformed input by rejecting the affected row and returning an issue record. It handles cancellation records by retaining them for cleaning statistics while excluding them from customer-value calculations. It refuses to cluster a population with fewer than two customers and caps the candidate cluster range for small populations. For eligible recommendations, a later failure in a real delivery adapter is outside the current process-local preview path; therefore the present system reports a safe preview rather than claiming delivery success.

TABLE VII
TRACEABILITY FROM BASELINE COMPONENT TO EVIDENCE

Component	Primary output	Verification evidence
Ingestion	Clean frame and rejection summary	Invalid dates, identifiers, quantities, prices, cancellations, and duplicates are tested.
RFM	Customer feature records	Hand-checked recency, distinct-invoice frequency, monetary totals, and quartile scores.
Clustering	K sweep, selected model, segments	Boundary cases, metric ranges, dynamic labels, and fixed-seed behavior are tested.
Fatigue	Ordered eligibility checks	Consent, suppression, frequency-cap, and recency failures are tested independently.
Recommendations	Campaign, score, rationale, audit data	Segment mapping, score bounds, explanation content, and ineligible outputs are tested.
Adapters	Simulated result and parsed events	Mock labels, signature parsing, and adapter-level duplicate detection are tested.

The principal failure mode for a future event path is duplicate or reordered input. Duplicate processing would inflate frequency and monetary value, potentially changing both the cluster and the campaign decision. Reordered processing can change a measured latency while not changing the source event time. The proposed event record therefore stores both timestamps and a persistent idempotency key. A second failure mode is model identity drift: K-Means cluster number 0 in one run is not necessarily the same behavioral group as cluster number 0 in another. Centroid alignment and label-comparison rules are therefore required before switching rates or ARI are interpreted.

B. Ethical and Operational Constraints

The system is a recommendation and audit prototype, not an autonomous permission to contact customers. Consent and suppression are hard gates. The default frequency cap is three messages in a rolling 30-day window. No bandit or stability optimization may trade away these constraints for lower latency, a higher score, or an apparent campaign benefit. The development adapter is mock-only by default, and the paper reports no live WhatsApp dispatch.

The research evaluation must also protect data minimisation. Customer identifiers used in examples should be synthetic or masked, event payloads should not include phone numbers, and exported experiment results should contain aggregate measures where individual records are not necessary. A production implementation would additionally require authentication, tenant-scoped authorisation, secrets management, retention rules, and an audit log that survives process restart. These are deployment requirements and are not silently assumed to be supplied by the current MVP.

C. Reproducibility Checklist

For each reported experiment, the following information should be committed with the output: dataset filename and cryptographic hash, raw and retained row counts, cancellation and duplicate counts, reference date, feature transformation, scaler configuration, candidate K range, random seed, package versions, CPU and memory environment, event replay order, micro-batch size, stability thresholds, and the exact code revision. The output should include both aggregate metrics and enough decision records to reproduce a switching-rate calculation. If a result is obtained from a separately downloaded UCI file, the download date, source URL, checksum, and preprocessing manifest must be added. Without this information, the result should remain labelled as unverified.

XI. Proposed Experiments and Pending Results

No event-to-profile latency, event-to-recommendation latency, throughput, switching rate, ARI, drift threshold, or stability overhead result is claimed in this draft. These results remain pending implementation and execution. Consent, suppression, and frequency-cap checks must be evaluated as invariants: no experiment may bypass them to obtain a performance result.

XII. Discussion

The implemented baseline establishes a useful control condition: it is simple, explainable, testable, and suitable for replay-based comparison. Its main research limitation is that it is batch-oriented and process-local. The planned extension can therefore test whether lower latency is obtained at the cost of unstable customer decisions, and whether stability checks make those changes more conservative and auditable.

A contextual bandit implementation and external order parsers are present as isolated services, but they are not treated as experimental results because they are not connected to the baseline recommendation route and do not receive durable feedback. Similarly, adapter-level webhook deduplication is not equivalent to a persistent event-ingestion system.

XIII. Limitations and Threats to Validity

The bundled dataset is small and synthetic, and its consent and engagement fields are designed test metadata rather than evidence of real customer behaviour. No UCI Online Retail file is checked into the repository, so claims based on the 541,909-row UCI dataset require a separately archived dataset, preprocessing manifest, and generated output. Internal clustering indices do not establish marketing effectiveness. Replay-based events do not reproduce network, queue, or source-system conditions. Small datasets can make K selection and stability estimates sensitive to preprocessing and random seed. Finally, the proposed materiality thresholds may encode policy choices and require sensitivity analysis.

XIV. Conclusion and Future Work

This paper defines a reproducible research extension for a working batch platform that combines RFM segmentation, explainable campaign recommendations, and mandatory marketing eligibility safeguards. The existing baseline is implemented and measured on the bundled verification dataset. The proposed contribution is an empirical comparison of batch, event-driven, and stability-aware decision paths, not a claim that the real-time system already exists. Future work will implement persistent event ingestion, affected-customer or micro-batch updates, cluster identity alignment, stability metrics, webhook processing, and experiments with latency and decision consistency.

References

[1] A. M. Hughes, *Strategic Database Marketing*. Chicago, IL, USA: Probus, 1994.

[2] J. MacQueen, "Some methods for classification and analysis of multivariate observations," in *Proc. Fifth Berkeley Symp. Math. Statist. Probability*, vol. 1, 1967, pp. 281–297.

[3] P. J. Rousseeuw, "Silhouettes: A graphical aid to the interpretation and validation of cluster analysis," *J. Comput. Appl. Math.*, vol. 20, pp. 53–65, 1987, doi: 10.1016/0377-0427(87)90125-7.

[4] D. L. Davies and D. W. Bouldin, "A cluster separation measure," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. PAMI-1, no. 2, pp. 224–227, 1979, doi: 10.1109/TPAMI.1979.4766909.

[5] T. Calinski and J. Harabasz, "A dendrite method for cluster analysis," *Commun. Statist. Theory Methods*, vol. 3, no. 1, pp. 1–27, 1974, doi: 10.1080/03610927408827101.

[6] D. Chen, S. L. Sain, and K. Guo, "Data mining for the online retail industry: A case study of RFM model-based customer segmentation using data mining," *J. Database Marketing Customer Strategy Manag.*, vol. 19, no. 3, pp. 197–208, 2012, doi: 10.1057/dbm.2012.17.

[7] P. S. Fader, B. G. S. Hardie, and K. L. Lee, "Counting your customers the easy way: An alternative to the Pareto/NBD model," *Marketing Sci.*, vol. 24, no. 2, pp. 275–284, 2005, doi: 10.1287/mksc.1040.0098.

[8] A. Godfrey, K. Seiders, and G. B. Voss, "Enough is enough! The compromising effect of relationship interactions on customer fatigue and satisfaction," *J. Marketing*, vol. 75, no. 4, pp. 110–128, 2011, doi: 10.1509/jmkg.75.4.110.

[9] V. Kumar and W. Reinartz, *Customer Relationship Management: Concept, Strategy, and Tools*, 3rd ed. Berlin, Germany: Springer, 2016, doi: 10.1007/978-3-662-55381-7.

[10] P. C. Verhoef, P. K. Kannan, and J. J. Inman, "From multi-channel retailing to omni-channel retailing," *J. Retailing*, vol. 91, no. 2, pp. 174–181, 2015, doi: 10.1016/j.jretai.2015.02.005.

[11] A. Bifet and R. Kirkby, "Data stream mining: A practical approach," in *ODP Technical Report*, 2009. [Online]. Available: https://moa.cms.waikato.ac.nz/

[12] C. C. Aggarwal, J. Han, J. Wang, and P. S. Yu, "A framework for clustering evolving data streams," in *Proc. 29th Int. Conf. Very Large Data Bases*, 2003, pp. 81–92.

[13] L. Hubert and P. Arabie, "Comparing partitions," *J. Classification*, vol. 2, pp. 193–218, 1985, doi: 10.1007/BF01908075.